

Joins continued: joins with **GROUP BY** and **CASE**

How do I get a correct total, and find what is missing?

BY THE END OF TODAY, YOU CAN

Class objectives

-
- 01 Count after LEFT JOIN; find unmatched rows
 - 02 Group by a cleaned column from another table
 - 03 Join four tables with GROUP BY, CASE, HAVING
 - 04 Stack with UNION ALL; count CROSS JOIN rows
 - 05 Explain why a bridge table adds rows
-

Warm-up

Five questions on D06, ungraded.

Suggested time • 10 minutes

Rows with no match

Counting after a LEFT JOIN, and finding what never matched.

COUNT(*) against COUNT(column) after a LEFT JOIN

Why a customer with no orders counts as 1.

Suggested time • 12 minutes

[labs/D07/01-Evr-Count-After-Left-Join/README.md](#)

COUNT(*) counts the empty row too

COUNT(o.order_id) skips the NULL that the LEFT JOIN added.

```
SELECT c.customer_id,  
       COUNT(*) AS rows_counted,  
       COUNT(o.order_id) AS orders  
FROM nakheel.customers AS c  
LEFT JOIN nakheel.orders AS o  
  ON c.customer_id = o.customer_id  
GROUP BY c.customer_id  
ORDER BY orders, c.customer_id;
```

→ **C-1013: 1 row counted, 0 orders**

Activity: Never ordered, never sold

In this activity, you will find the customers who never ordered and the products never sold, with LEFT JOIN and IS NULL.

Suggested time

10 minutes

[labs/D07/02-Stu-Never-Ordered/README.md](#)

Activity: Never ordered, never sold

INSTRUCTIONS

1. Which customers signed up but never placed an order?
 2. Which products have never sold?
- Before each: which table goes on the left?

HINT

After a LEFT JOIN, test the right-hand table's key: `o.order_id IS NULL`.

BONUS

How many never-ordered customers are in each city, as typed?

Time's up. Let's review.

Never ordered, never sold

Solved: `labs/D07/02-Stu-Never-Ordered/Solved/never_ordered.sql`

The join family: RIGHT, FULL and the no-match joins

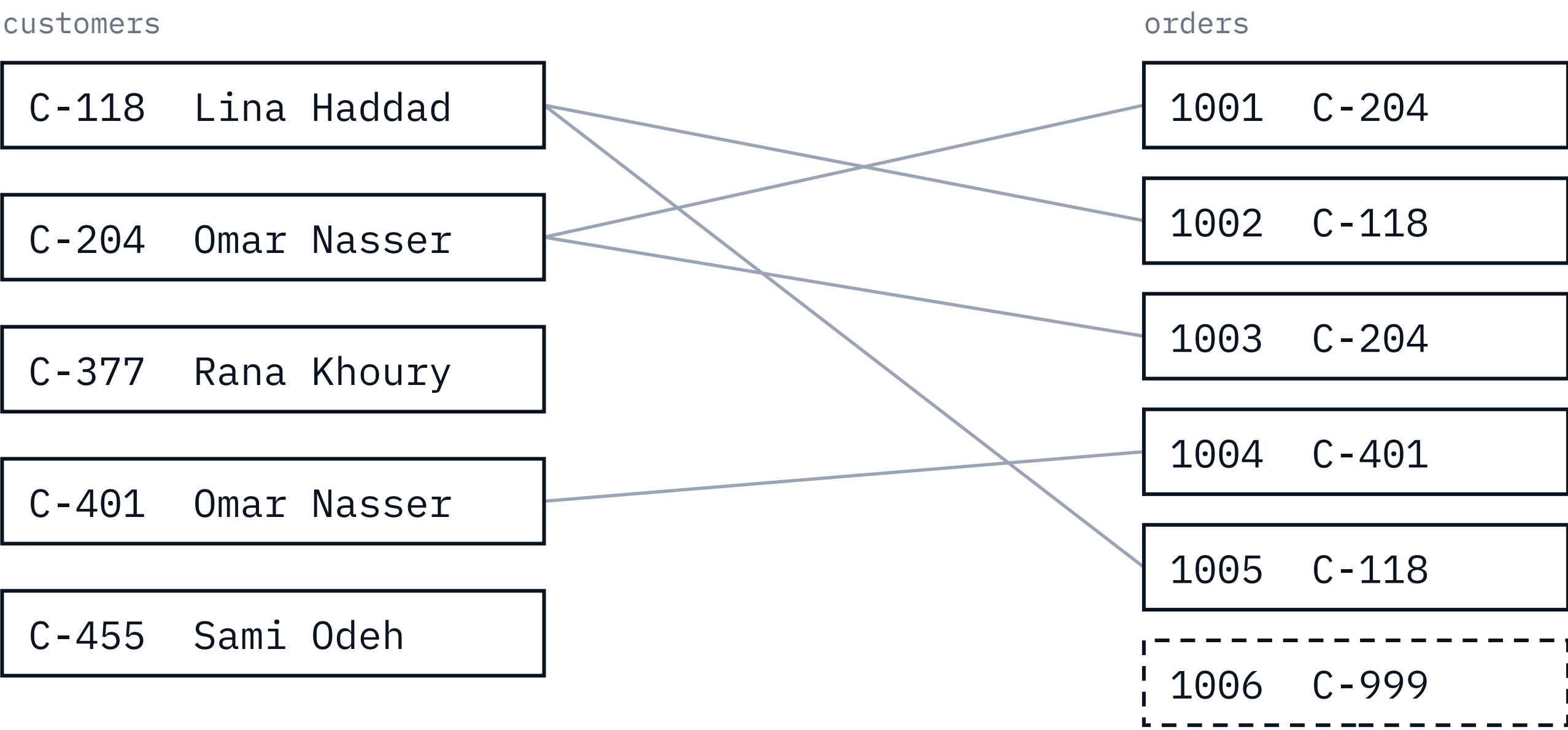
Which rows each join keeps, drawn two ways. You will read these more often than you write them.

Suggested time • 10 minutes

[labs/D07/03-Ins-Join-Family/README.md](https://github.com/DataCamp/labs/D07/03-Ins-Join-Family/README.md)

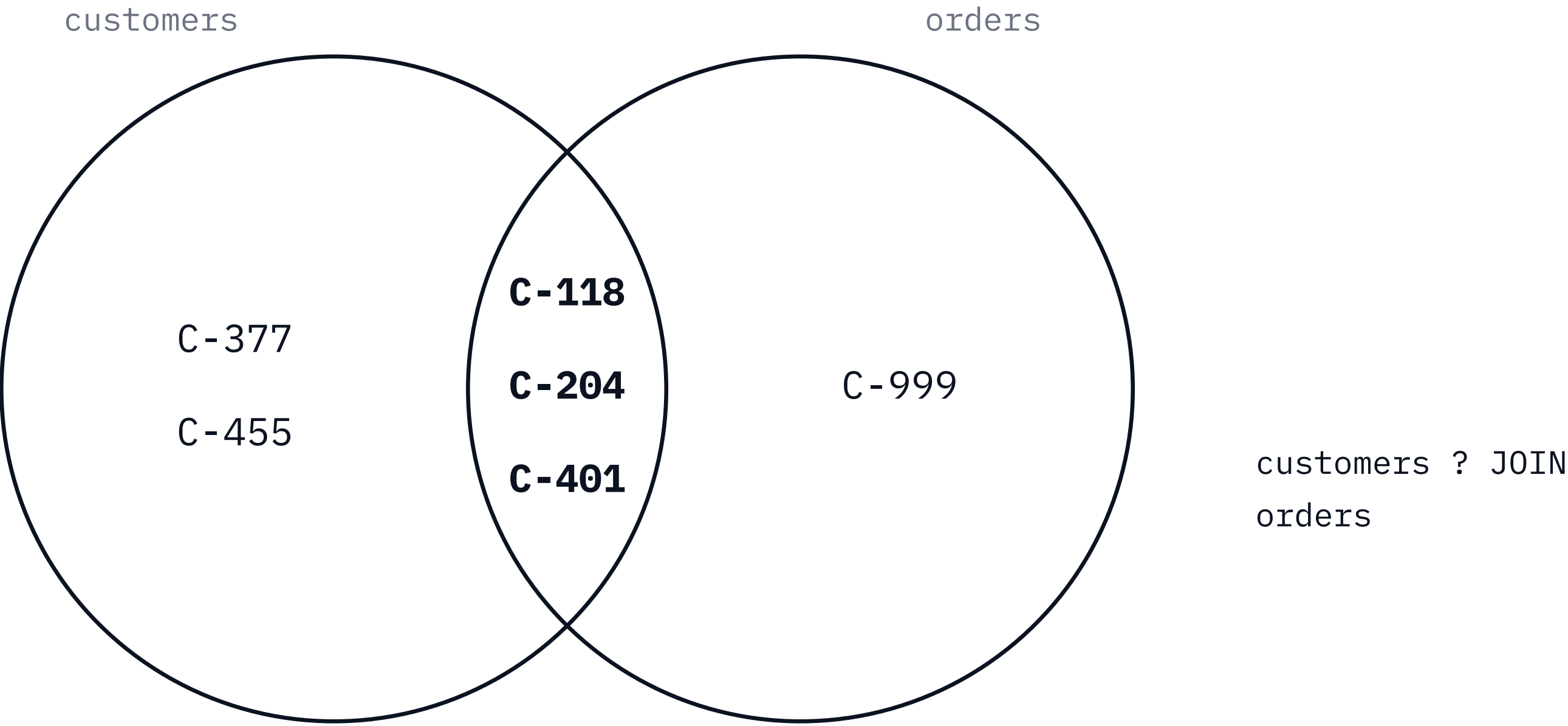
Add one order with no customer

Order 1006 points to C-999, who is not in customers. Nakheel has a real one: 101664.



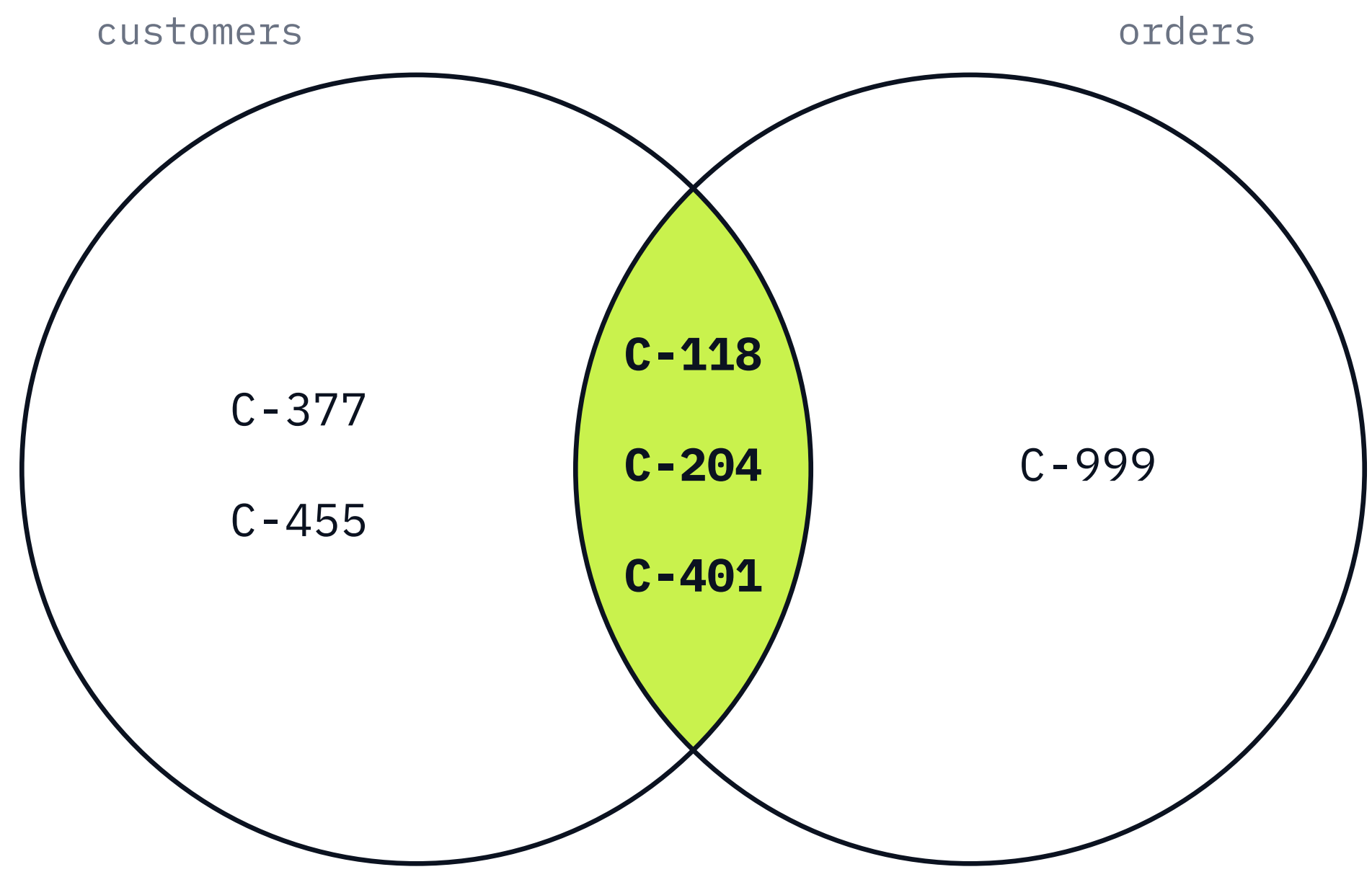
Which keys sit on each side

Each label is a customer_id value, not a row. The overlap holds the keys found in both tables.



INNER JOIN keeps the overlap

Only keys found in both tables come back.



ROWS · D03 + 1006

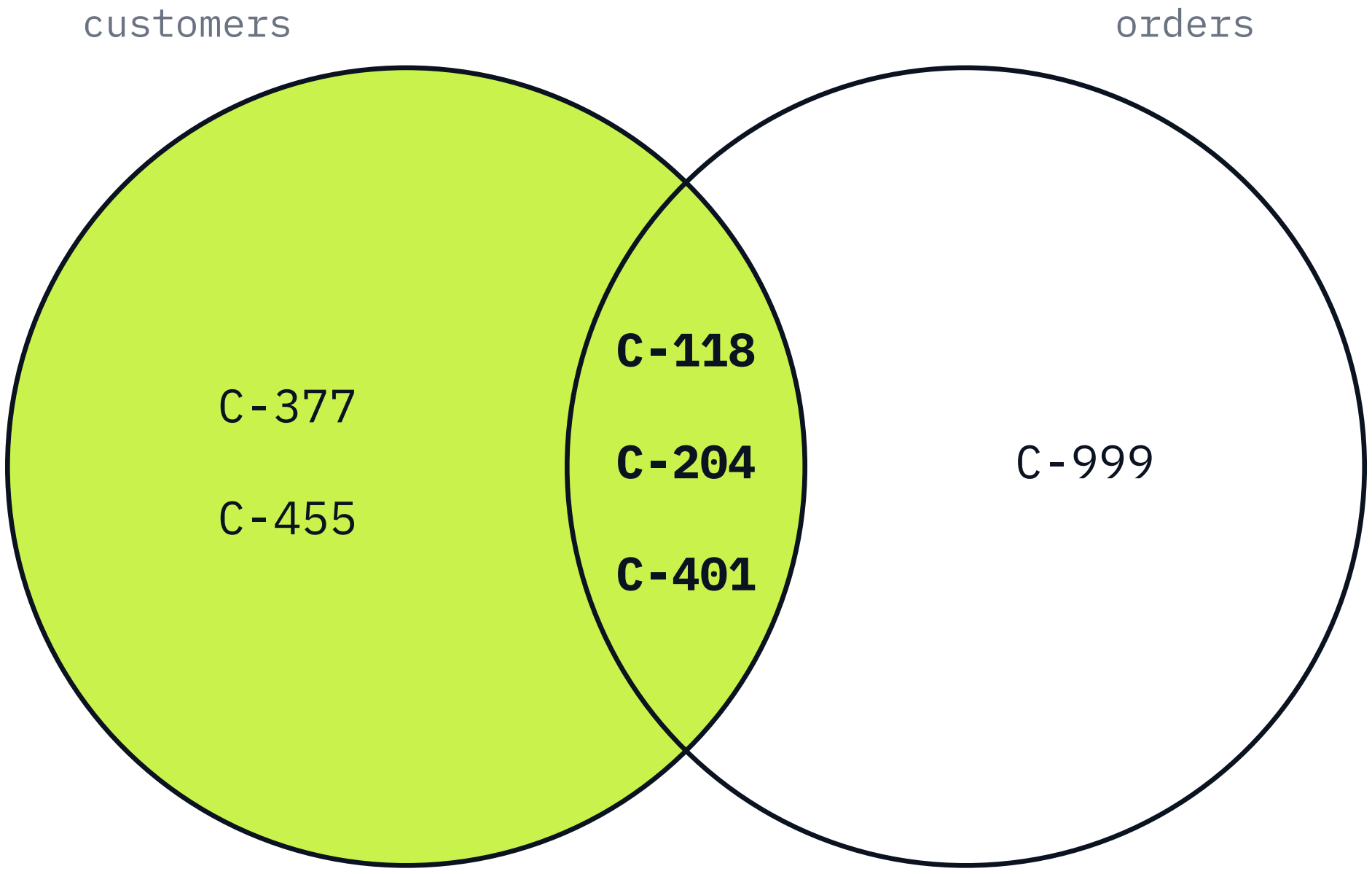
5

Nakheel: 2,999

customers
INNER JOIN orders

LEFT JOIN keeps the whole left circle

Every customer, with NULL where there is no order.



ROWS · D03 + 1006

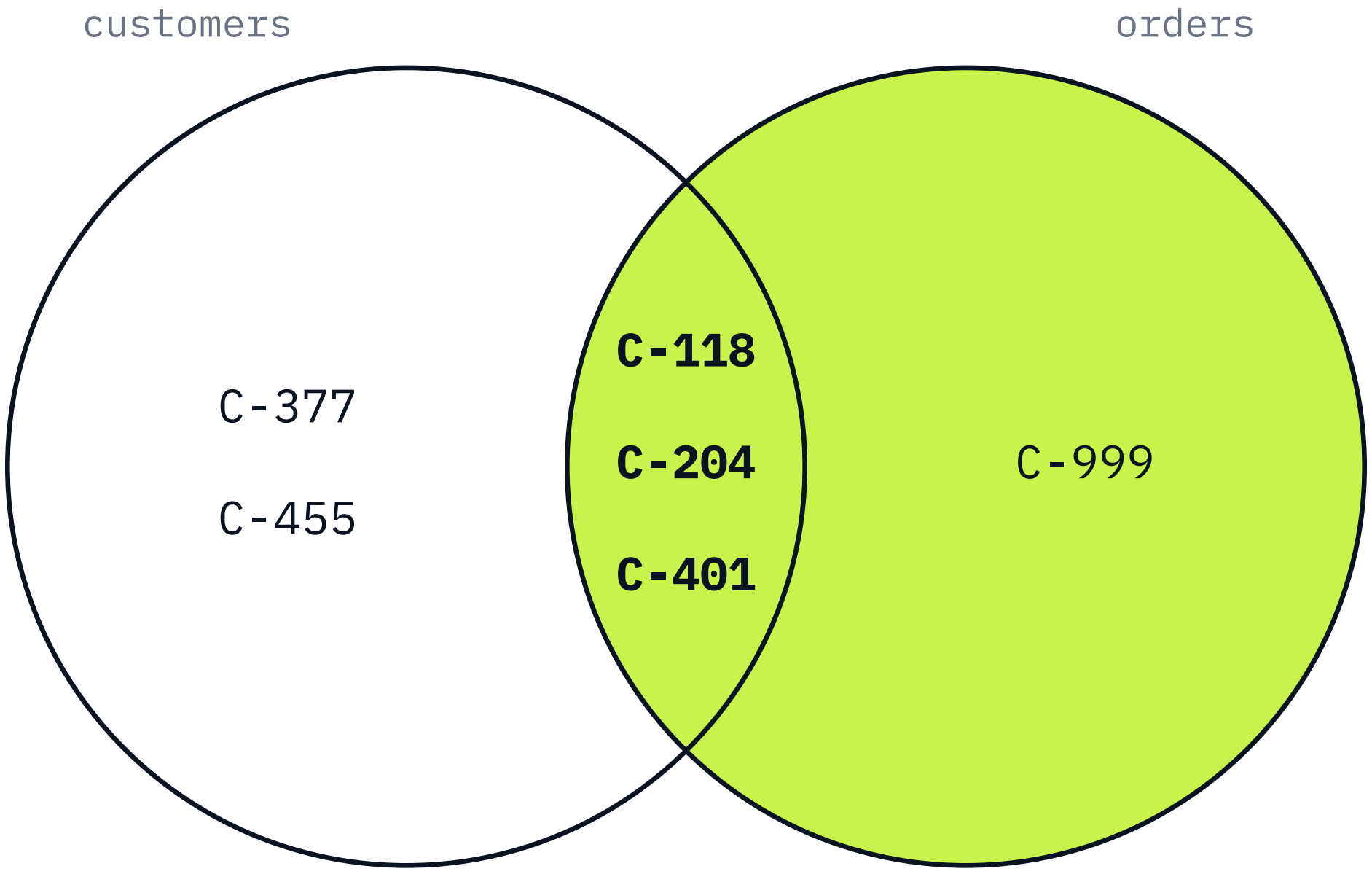
7

Nakheel: 3,038

customers
LEFT JOIN orders

RIGHT JOIN keeps the whole right circle

Every order, with NULL where there is no customer. Swap the tables and it is a LEFT JOIN.



ROWS · D03 + 1006

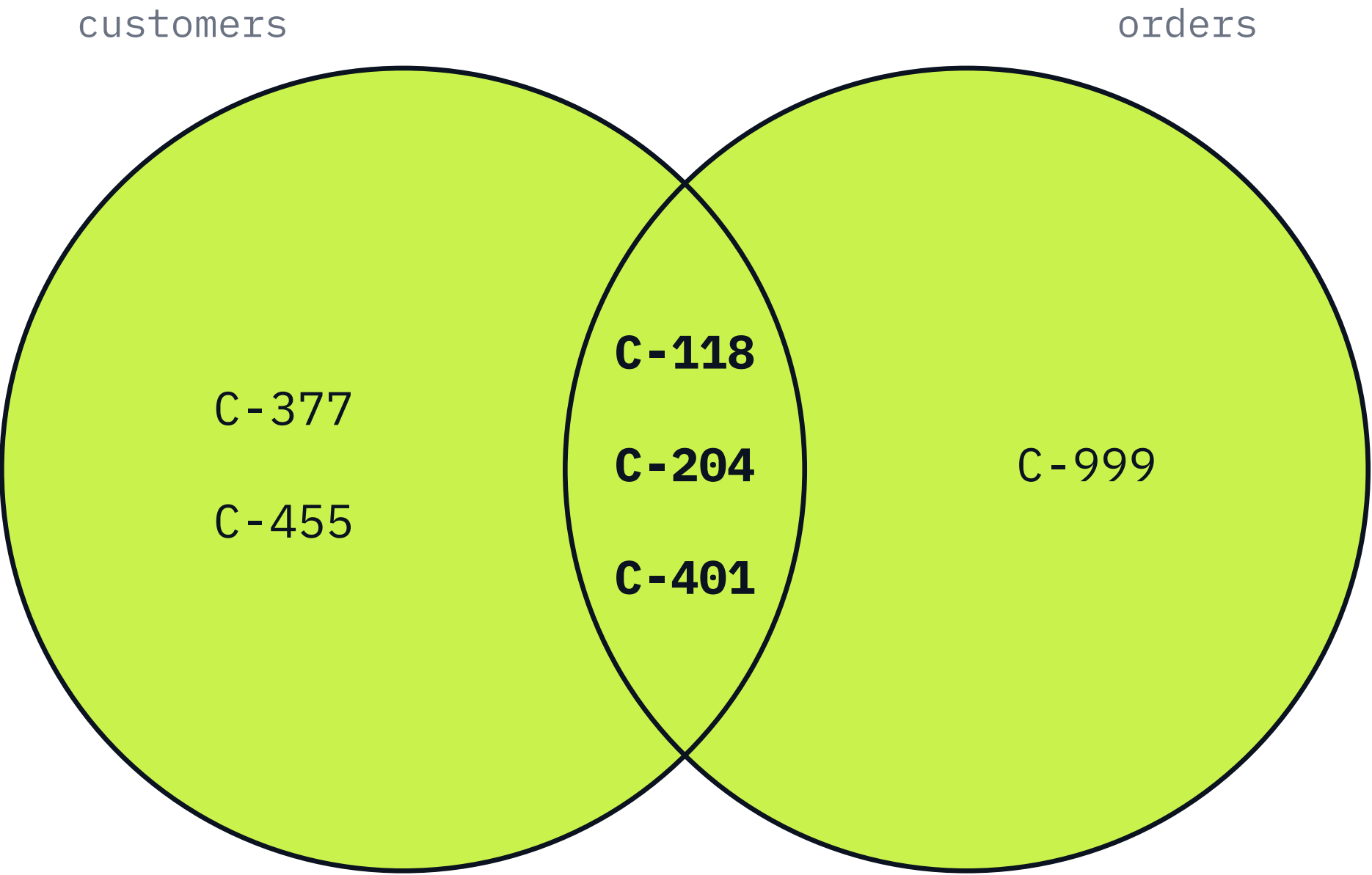
6

Nakheel: 3,000

customers
RIGHT JOIN orders

FULL OUTER JOIN keeps both circles

Every row from each side, matched where it can be.



ROWS · D03 + 1006

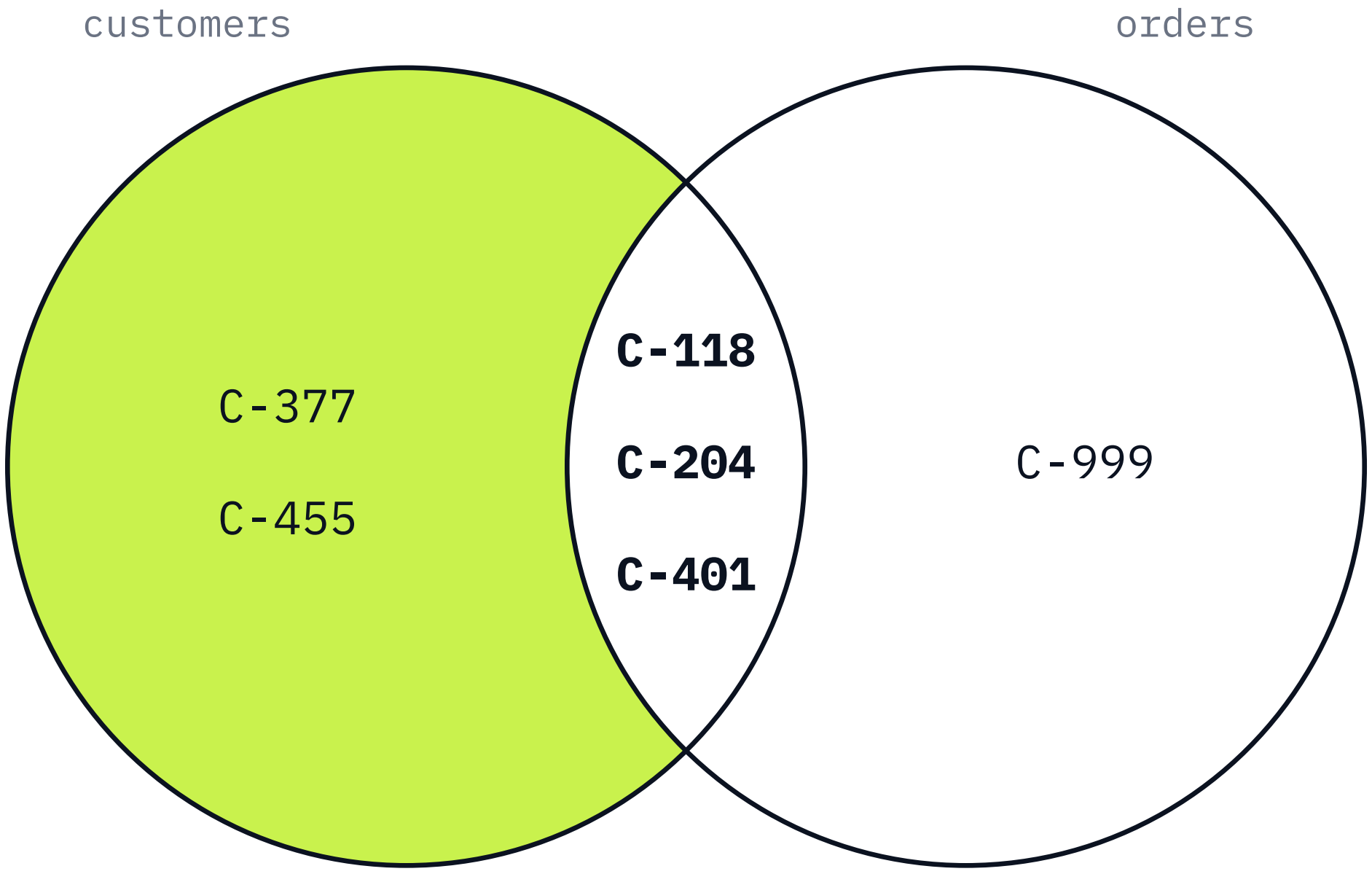
8

Nakheel: 3,039

customers
FULL OUTER JOIN
orders

No match: LEFT JOIN, then IS NULL

The left crescent is the never-ordered list from 2.2. Power Query calls it a left anti join.



ROWS · D03 + 1006

2

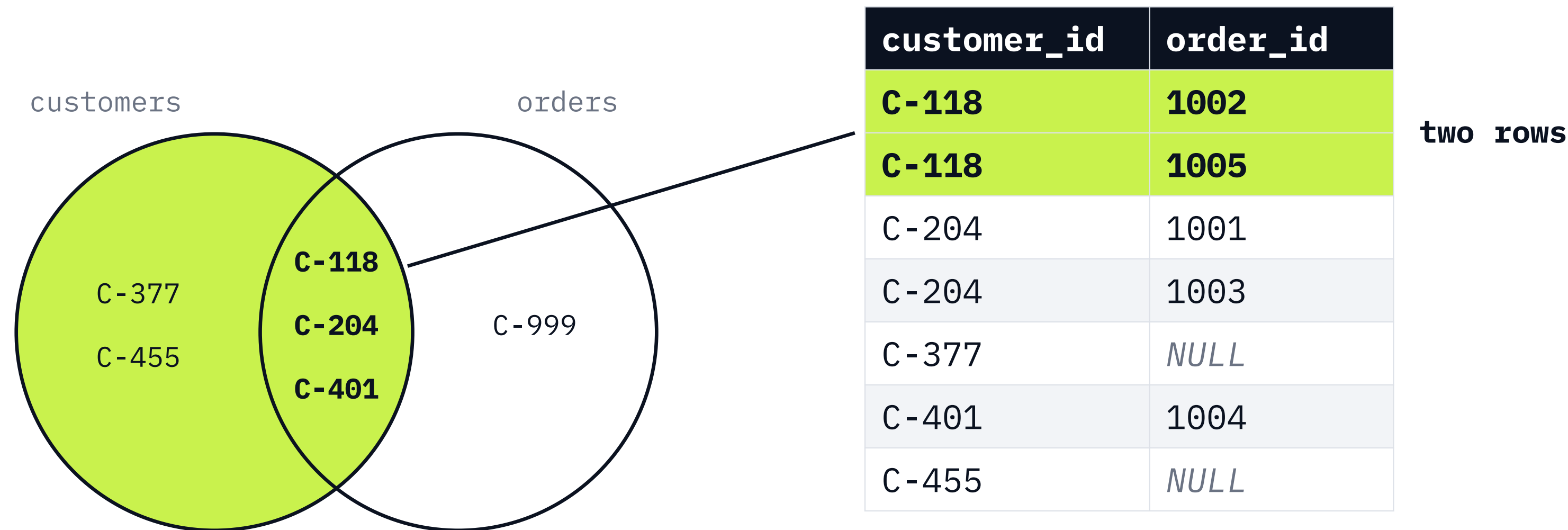
Nakheel: 39

```
LEFT JOIN orders
WHERE o.order_id
IS NULL
```

WHERE THE CIRCLES STOP

The circles hide the copies

C-118 is one key in the circles but two rows after the LEFT JOIN. Circles show which keys survive; count rows with the lines.



Same two tables, four results

The five matched rows are in every result. Each join adds a different set of unmatched rows, lit.

INNER • 5 rows

customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004

LEFT • 7 rows

customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004
C-377	NULL
C-455	NULL

RIGHT • 6 rows

customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004
NULL	1006

FULL • 8 rows

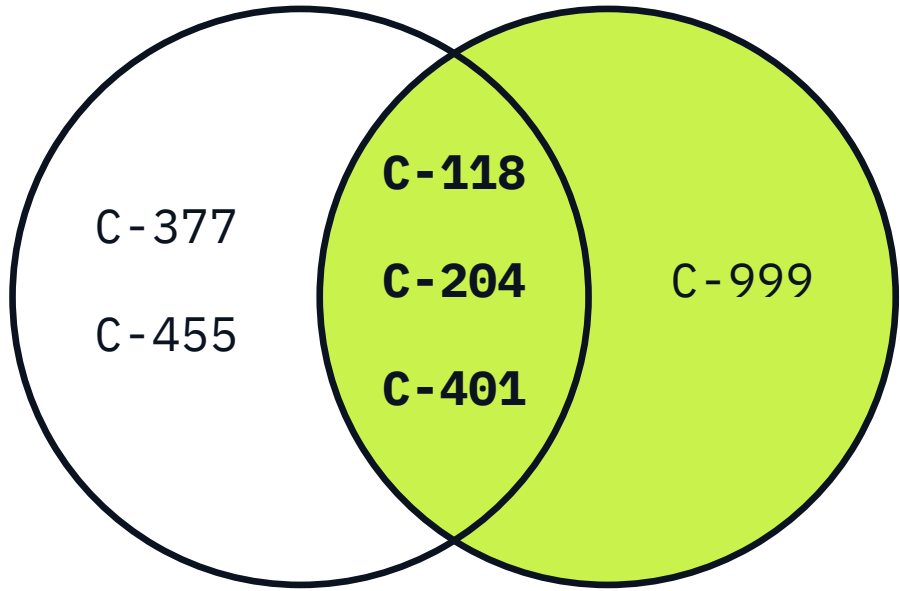
customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004
C-377	NULL
C-455	NULL
NULL	1006

RIGHT JOIN: every order, matched or not

D03.customers

customer_id	customer_name
C-118	Lina Haddad
C-204	Omar Nasser
C-377	Rana Khoury
C-401	Omar Nasser
C-455	Sami Odeh

customers



orders

D03.orders

order_id	customer_id
1001	C-204
1002	C-118
1003	C-204
1004	C-401
1005	C-118
1006	C-999

result · 6 rows

```
SELECT c.customer_id, c.customer_name,  
       o.order_id  
FROM D03.customers AS c  
RIGHT JOIN D03.orders AS o  
ON c.customer_id = o.customer_id;
```

customer_id	customer_name	order_id
C-118	Lina Haddad	1002
C-118	Lina Haddad	1005
C-204	Omar Nasser	1001
C-204	Omar Nasser	1003
C-401	Omar Nasser	1004
NULL	NULL	1006

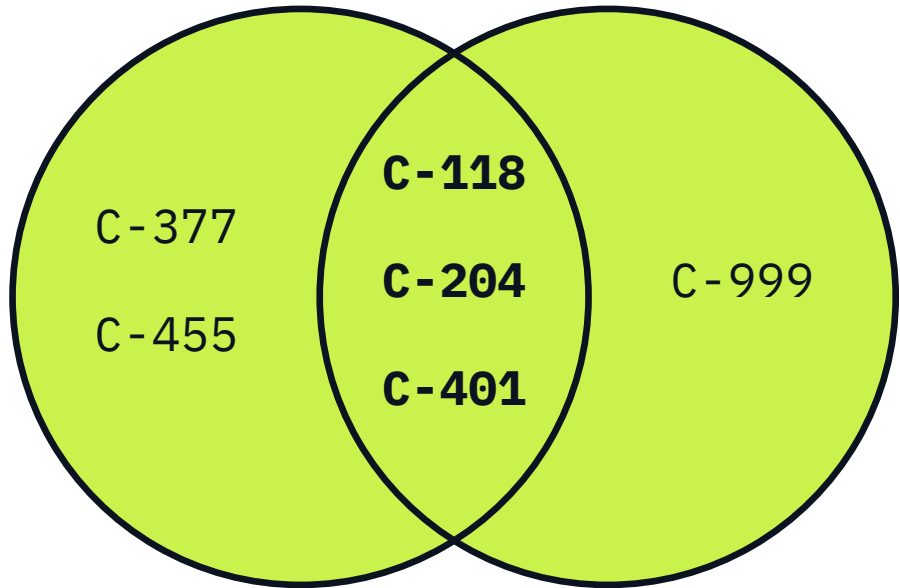
Rows out = matches + orders with no customer: **5 + 1 = 6.**

FULL OUTER JOIN: every row from both sides

D03.customers

customer_id	customer_name
C-118	Lina Haddad
C-204	Omar Nasser
C-377	Rana Khoury
C-401	Omar Nasser
C-455	Sami Odeh

customers



orders

D03.orders

order_id	customer_id
1001	C-204
1002	C-118
1003	C-204
1004	C-401
1005	C-118
1006	C-999

```
SELECT c.customer_id, c.customer_name,
       o.order_id
FROM D03.customers AS c
FULL OUTER JOIN D03.orders AS o
ON c.customer_id = o.customer_id;
```

Rows out = 5 matches + 2 customers + 1 order = 8.

result · 8 rows

customer_id	customer_name	order_id
C-118	Lina Haddad	1002
C-118	Lina Haddad	1005
C-204	Omar Nasser	1001
C-204	Omar Nasser	1003
C-377	Rana Khoury	NULL
C-401	Omar Nasser	1004
C-455	Sami Odeh	NULL
NULL	NULL	1006

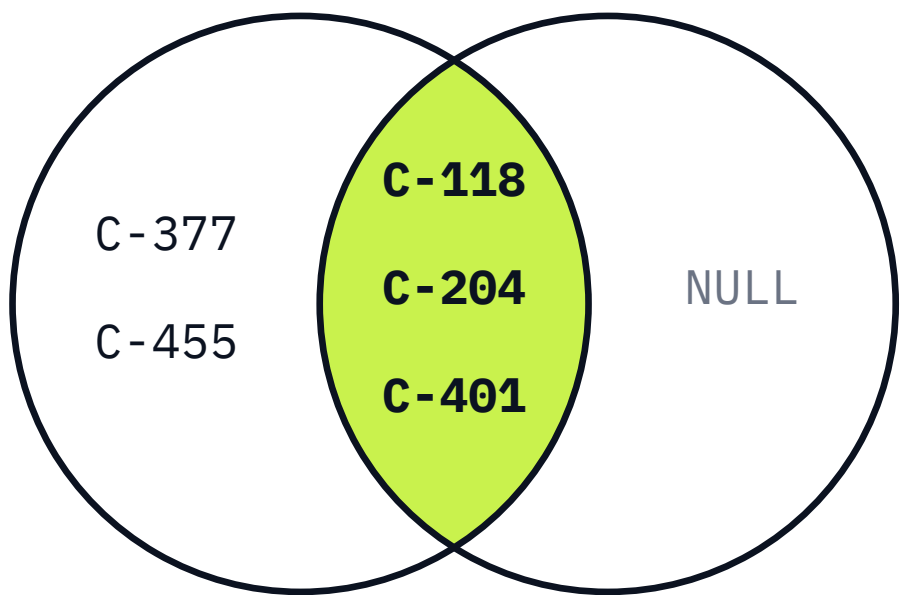
A NULL key matches nothing

D03.customers

customer_id	customer_name
C-118	Lina Haddad
C-204	Omar Nasser
C-377	Rana Khoury
C-401	Omar Nasser
C-455	Sami Odeh

customers

orders



D03.orders

order_id	customer_id
1001	C-204
1002	C-118
1003	C-204
1004	C-401
1005	C-118
1007	NULL

result · 5 rows

customer_id	customer_name	order_id
C-118	Lina Haddad	1002
C-118	Lina Haddad	1005
C-204	Omar Nasser	1001
C-204	Omar Nasser	1003
C-401	Omar Nasser	1004

```
SELECT c.customer_id, c.customer_name,
       o.order_id
FROM D03.customers AS c
INNER JOIN D03.orders AS o
ON c.customer_id = o.customer_id;
```

NULL = NULL is not TRUE, so 1007 matches nothing. Find it with **IS NULL**.

Two orders, no name, two different reasons

Every order is kept, so both come back with NULL for the name. Only o.customer_id says why.

```
SELECT o.order_id, o.customer_id, c.customer_name
FROM D03.orders AS o
LEFT JOIN D03.customers AS c
  ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

- 1006 • C-999 a customer missing from customers
- 1007 • NULL no customer ever recorded

order_id	o.customer_id	customer_name
1001	C-204	Omar Nasser
1002	C-118	Lina Haddad
1003	C-204	Omar Nasser
1004	C-401	Omar Nasser
1005	C-118	Lina Haddad
1006	C-999	NULL
1007	NULL	NULL

Same NULL name, different fixes: a missing customer to find, or a guest order to label.

Joins meet GROUP BY and CASE

The join brings the column; D04 does the rest.

Group by a column from the other table

Completed revenue by customer segment: audit first, then group.

Suggested time • 12 minutes

[labs/D07/04-Ins-Segment-After-Join/README.md](#)

INNER JOIN loses one order

Completed orders, on orders

2,771

316,023.50

After INNER JOIN customers

2,770

315,085.50

Order 101664 and 938.00 are gone: the orphan from D06. So LEFT JOIN.

LEFT JOIN brings the segment

LEFT JOIN keeps every order, including 101664 with no customer.

```
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,  
       COUNT(*) AS completed_orders,  
       ROUND(SUM(o.order_total), 2) AS revenue  
FROM nakheel.orders AS o  
LEFT JOIN nakheel.customers AS c  
  ON o.customer_id = c.customer_id  
WHERE o.status = 'completed'  
GROUP BY segment_clean  
ORDER BY revenue DESC;
```

Completed orders only

WHERE runs after the join and before any grouping.

```
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,  
       COUNT(*) AS completed_orders,  
       ROUND(SUM(o.order_total), 2) AS revenue  
FROM nakheel.orders AS o  
LEFT JOIN nakheel.customers AS c  
  ON o.customer_id = c.customer_id  
WHERE o.status = 'completed'  
GROUP BY segment_clean  
ORDER BY revenue DESC;
```

Clean the segment with D04's functions

COALESCE gives the order with no customer a label instead of a blank group.

```
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,  
       COUNT(*) AS completed_orders,  
       ROUND(SUM(o.order_total), 2) AS revenue  
FROM nakheel.orders AS o  
LEFT JOIN nakheel.customers AS c  
  ON o.customer_id = c.customer_id  
WHERE o.status = 'completed'  
GROUP BY segment_clean  
ORDER BY revenue DESC;
```

Group by the cleaned segment

In BigQuery the SELECT alias can be used in GROUP BY.

```
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,  
       COUNT(*) AS completed_orders,  
       ROUND(SUM(o.order_total), 2) AS revenue  
FROM nakheel.orders AS o  
LEFT JOIN nakheel.customers AS c  
  ON o.customer_id = c.customer_id  
WHERE o.status = 'completed'  
GROUP BY segment_clean  
ORDER BY revenue DESC;
```

→ 3 rows: retail · wholesale · unknown

THE GROUPS ADD UP TO THE AUDIT

Wholesale: 377 orders, nearly half the revenue

Add the three revenues: 316,023.50, the audit total. No row was lost or copied.

segment_clean	completed_orders	revenue
retail	2,393	160,881.50
wholesale	377	154,204.00
unknown	1	938.00
total	2,771	316,023.50

A cleaning rule of your own

Completed revenue by city: 8 cities, but 11 rows. Write the rule yourself.

Suggested time • 12 minutes

[labs/D07/05-Evr-City-Rule/README.md](https://github.com/DataCamp/labs/blob/main/D07/05-Evr-City-Rule/README.md)

Write the rule in words first

-
- 01 No city at all → unknown
 - 02 Al Zarqa → zarqa
 - 03 Everything else → trimmed and lower-case
-

The rule, line for line

Without the NULL line, ELSE returns NULL, not 'unknown'. Test for NULL first, as a habit.

```
SELECT CASE
    WHEN c.city IS NULL THEN 'unknown'
    WHEN LOWER(TRIM(c.city)) = 'al zarqa' THEN 'zarqa'
    ELSE LOWER(TRIM(c.city))
END AS city_clean,
COUNT(*) AS completed_orders,
ROUND(SUM(o.order_total), 2) AS revenue
FROM nakheel.orders AS o
LEFT JOIN nakheel.customers AS c
    ON o.customer_id = c.customer_id
WHERE o.status = 'completed'
GROUP BY city_clean
ORDER BY revenue DESC;
```

→ 9 rows

amman first
1,064 orders
130,848.00

unknown
37 orders
3,197.00

STUDENT DO

Activity: Category by segment, four tables

In this activity, you will join four tables to report completed units and revenue by category and segment.

Suggested time

18 minutes

[labs/D07/06-Stu-Category-By-Segment/README.md](#)

Activity: Category by segment, four tables

INSTRUCTIONS

1. Audit: rows and line value after all three joins.
 2. Units and revenue by category and cleaned segment.
 3. Keep only the combinations worth more than 20,000.
- Start from order_items: it has the amounts.

HINT

Which table has the category? The segment? Line → order → customer.

BONUS

Early finishers: the Bonus in the README.

Time's up. Let's review.

Category by segment

Solved: `labs/D07/06-Stu-Category-By-Segment/Solved/category_by_segment.sql`

Stack and pair: UNION and CROSS JOIN

The two other ways to combine data.

UNION ALL and UNION DISTINCT

A join goes across; UNION goes down. One D03 example with both kinds of repeat.

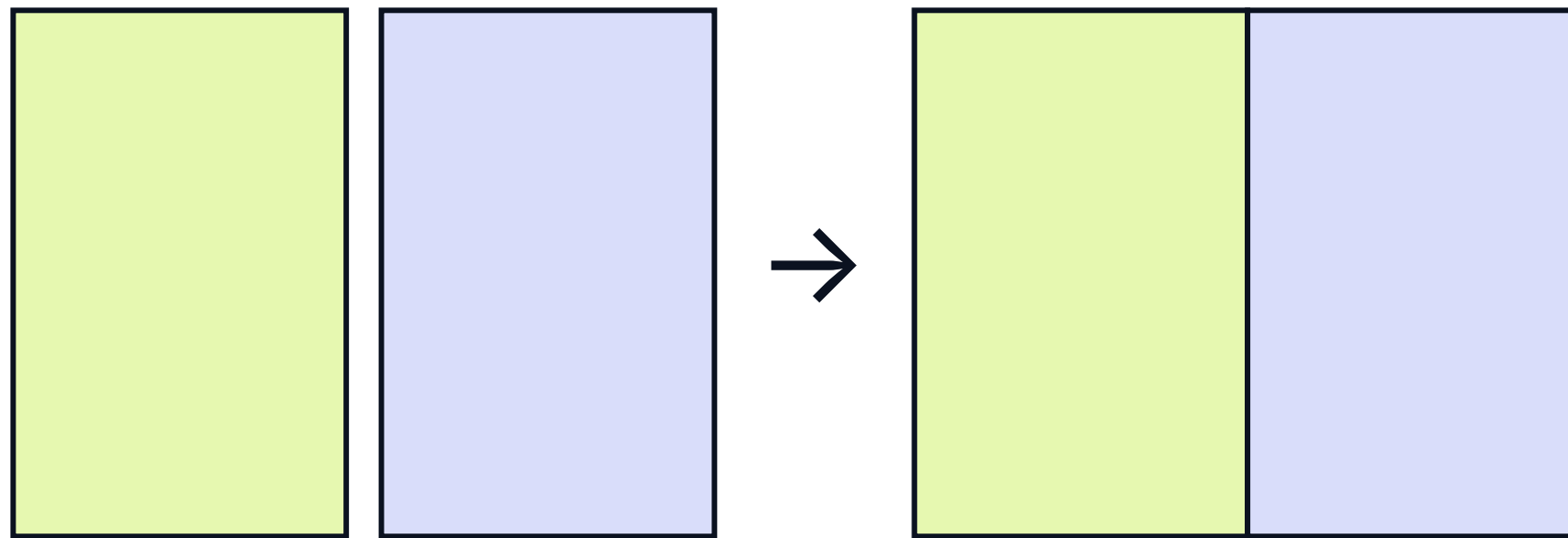
Suggested time • 8 minutes

[labs/D07/07-Ins-Union-and-Cross-Join/README.md](#)

JOIN adds columns. UNION adds rows.

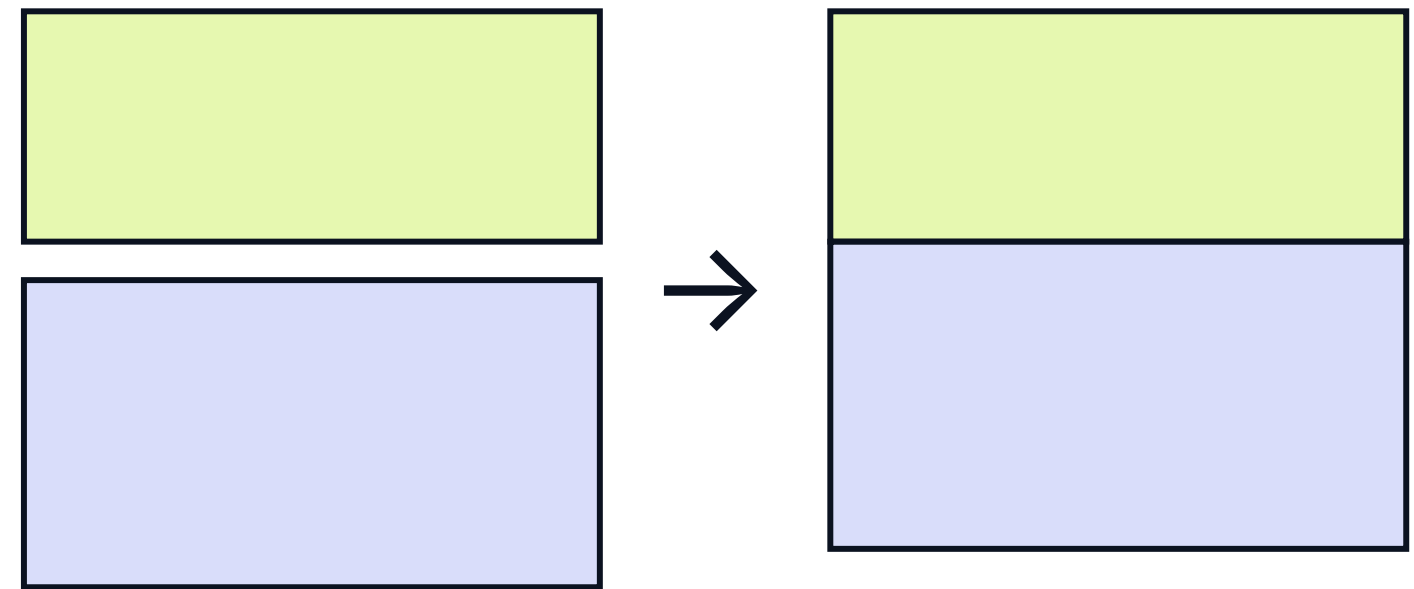
A join puts two tables side by side on a key. UNION stacks one result under another.

JOIN • goes across



more columns • rows matched on a key

UNION • goes down



more rows • same columns, no key

DEFINITION

UNION stacks the rows of two queries into one result. Both return the same number of columns, in the same order, with matching types.

Two lists of customer IDs

Q1: customers in Amman. Q2: the customer on each completed order.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118

UNION ALL keeps every row

Q2's rows go under Q1's. The count is always rows in Q1 + rows in Q2.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118



UNION ALL

customer_id
C-118
C-377
C-204
C-204
C-401
C-118

ROWS

6

2 + 4: nothing removed

Two kinds of repeat

C-118 is in both queries. C-204 repeats inside Q2, once per order.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118



UNION ALL

customer_id
C-118
C-377
C-204
C-204
C-401
C-118

← C-118: in Q1 and in Q2

← C-204: twice inside Q2

← C-118 again

UNION DISTINCT keeps each row once

DISTINCT removes the repeats inside each query as well as between them.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118



UNION DISTINCT

customer_id
C-118
C-204
C-377
C-401

ROWS

4

each value once
sorted with ORDER BY

BigQuery makes you choose ALL or DISTINCT

Most databases read a bare UNION as DISTINCT. BigQuery stops with a syntax error.

```
SELECT customer_id  
FROM D03.customers  
WHERE city = 'Amman'  
UNION  
SELECT customer_id  
FROM D03.orders  
WHERE status = 'completed';
```

Syntax error: Expected keyword ALL or keyword DISTINCT but got keyword SELECT

Stack data with UNION ALL, then check rows out = rows in Q1 + rows in Q2. Use UNION DISTINCT when you want a list of different values.

The same stack in Power BI

Power Query’s Append keeps every row, like UNION ALL, but lines columns up by name. SQL and DAX go by position.

	Keeps repeats	Matches columns by	Names from
SQL UNION ALL	yes	position	first query
SQL UNION DISTINCT	no	position	first query
Power Query Append	yes	name	all tables
DAX UNION()	yes	position	first table

CROSS JOIN

Every row paired with every row: the grid that every join starts from.

Suggested time • 7 minutes

[labs/D07/07-Ins-Union-and-Cross-Join/README.md](#)

DEFINITION

CROSS JOIN pairs every row of one table with every row of the other. There is no ON: rows out = rows in A $\times$ rows in B.

5 customers × 5 orders = 25 rows

Each cell is one output row: a customer paired with an order, whether the keys match or not.

	1001 C-204	1002 C-118	1003 C-204	1004 C-401	1005 C-118	ROWS
C-118						25
C-204						
C-377						
C-401						
C-455						

The matching cells are the INNER JOIN

5 cells have the same customer_id on both sides. Every join keeps part of this grid.

	1001 C-204	1002 C-118	1003 C-204	1004 C-401	1005 C-118	ROWS
C-118						5
C-204						
C-377						
C-401						
C-455						

LEFT JOIN keeps the rows with no cell

C-377 and C-455 have no lit cell, so LEFT JOIN keeps each of them once, with NULL.

	1001 C-204	1002 C-118	1003 C-204	1004 C-401	1005 C-118		ROWS
C-118							7
C-204							
C-377						+1 row, NULL	
C-401							
C-455						+1 row, NULL	

At Nakheel, the grid has 1.5 million rows

customers	× orders	= CROSS JOIN rows
500	3,000	1,500,000

A join on a column that repeats on both sides, such as a name or a city, grows toward this. Check 3 of the Five Checks catches it before you run.

Every customer against every category

CROSS JOIN gives every pair a row, even with no sale. Translate: which customers have never bought Outerwear?

	Shoes	Accessories	Outerwear	ROWS
C-118 Lina Haddad	bought	bought	0	15
C-204 Omar Nasser	bought	bought	0	
C-377 Rana Khoury	0	0	0	
C-401 Omar Nasser	0	bought	bought	
C-455 Sami Odeh	0	0	0	
				9 pairs: no sale 4: no Outerwear

A bridge table, in a new sector

Pagila films: a join that multiplies rows on purpose.

A bridge table: films per category

Pagila, a DVD rental shop's sample database: a new sector, the same method.

Suggested time • 28 minutes

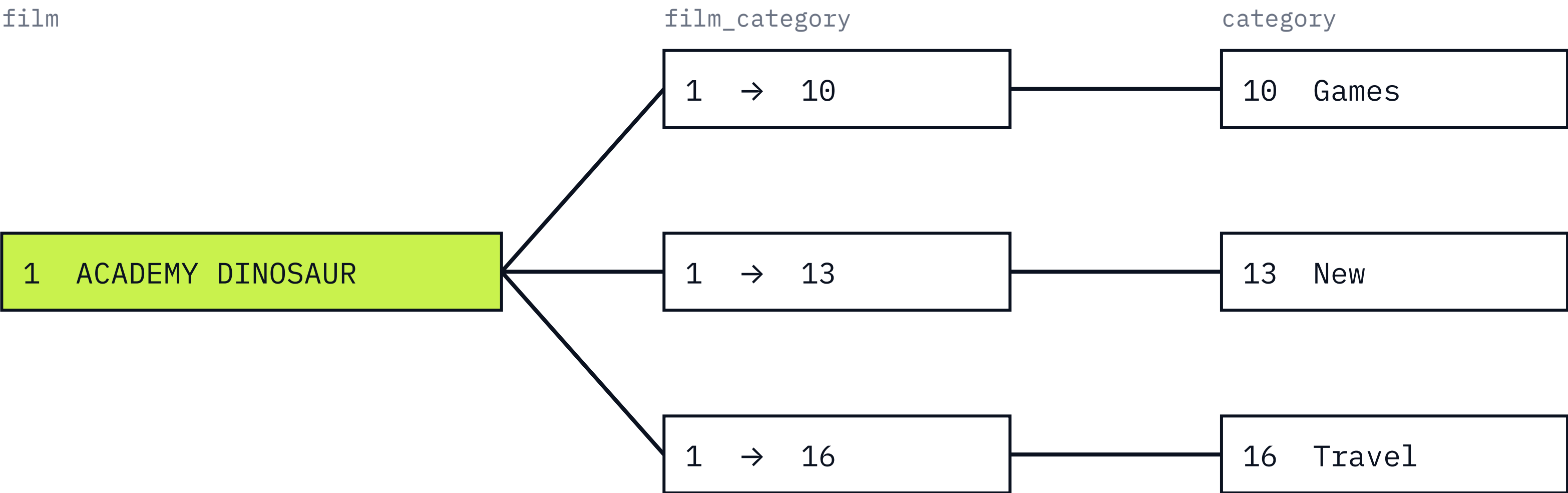
[labs/D07/08-Evr-Films-Per-Category/README.md](#)

DEFINITION

A bridge table sits between two tables and holds one row per link: in Pagila, one row per film per category.

One film, three categories

film_category holds one row per film per category, so film 1 comes back three times.



Time's up. Let's review.

Why 1,000 films become 2,367 rows

Solved: `labs/D07/08-Evr-Films-Per-Category/Solved/films_per_category.sql`

1,000 films, 2,367 rows

film

1,000

One row per film

film JOIN film_category

2,367

One row per film per category

36 films have 1 category, 561 have 2, 403 have 3. SUM(rental_rate) by category gives 7,077.33, not 2,980.00.

Break

 Back at 7:00 PM

After the break: the joined-report lab and a review of C1. Ask in your group channel.

Lab: one joined report

Everything from D06 and D07 in one report, with an audit.

Launch the lab

One report that uses the whole of D06 and D07, with an audit before any grouping.

Suggested time • 5 minutes

[labs/D07/09-Lab-Joined-Report/README.md](#)

Lab: A joined report with an audit

In this lab, you will build one Nakheel report: segment and channel, size bands, and everything with no match.

Suggested time

35 minutes

[labs/D07/09-Lab-Joined-Report/README.md](#)

Lab: A joined report with an audit

CHECKPOINTS

- C1 Audit, then completed revenue by cleaned segment and channel.
- C2 Add the D04 size band; keep groups over 20,000 with HAVING.
- C3 Never ordered and never sold, in one list with UNION ALL.

HINT

Audit before any grouping: rows, different orders, total.

TIMING

C2 in class if there is time. C3 after class.

Time's up. Let's review.

The C1 audit

Solved: `labs/D07/09-Lab-Joined-Report/Solved/joined_report.sql` · after class

LEFT JOIN customers: nothing lost, nothing copied

Every report today had the same check. Then show the C1 result: 5 rows.

	Rows	Different orders	SUM(order_total)
orders, completed	2,771	2,771	316,023.50
LEFT JOIN customers	2,771	2,771	316,023.50

Join reference card

Customers written first. Counts from the course data. UNION ALL is Power Query’s Append.

Join	Keeps	D03	Nakheel	Power Query
INNER JOIN	rows with a match on both sides	5	2,999	Inner
LEFT JOIN	every left row, NULL if no match	7	3,038	Left outer
RIGHT JOIN	every right row, NULL if no match	5	3,000	Right outer
FULL OUTER JOIN	every row from both sides	7	3,039	Full outer
LEFT JOIN + IS NULL	left rows with no match	2	39	Left anti
RIGHT JOIN + IS NULL	right rows with no match	0	1	Right anti
CROSS JOIN	every pair, no ON	25	1,500,000	(none)

CLOSE

What you can do now

- Join four tables and prove the total
- Clean with CASE; test groups with HAVING
- Find unmatched rows

NEXT • D08

The category-by-segment query was long. How would you break it into named steps, saved for Power BI?